

DSPy_Shizuoka_V3 仕様書（最終版・再現用）

最適化アルゴリズム自動生成パイプライン

DSPy_Shizuoka_V3 プロジェクト

2026 年 8 月 4 日

概要

本書は、DSPy + GEPA により最適化問題の要件テキストから実行可能な Python 関数 `solve(instance)` を自動生成・評価・改良するパイプライン「DSPy_Shizuoka_V3」の**最終版仕様**を、別部門が本ソースと同一データを用いて同じ実験を再現できることを目的として記述したものである。アルゴリズムの詳細、評価メトリックの数式、Signature 指示文、Demonstrations、学習設定、再現手順、および参照値（OR-Tools 等による既知解）との精度比較を含む。コードは付属の `src/` 配下および `train_gepa_v3.py` を参照する。本書には開発途上の「フェーズ」履歴は含めず、現行の到達点のみを記す。

目次

1	目的とスコープ	1
2	実行環境と依存関係	2
3	データ仕様	2
3.1	入力ファイル形式	2
3.2	訓練/テスト分割	3
3.3	要件テキストの構築	3
4	パイプライン構成	3
4.1	アルゴリズム生成 (AlgorithmGenerator)	3
4.2	安全実行 (safe_exec)	4
5	Signature 指示文 (初期)	4
6	評価メトリック	4
6.1	目的フィールドの自動検出	4
6.2	スコア計算 (compute_v3_score)	5
6.3	status 一覧	6
6.4	フィードバック生成	6
7	参照フリーモードの厳密化	6
8	Demonstrations と学習設定	6
8.1	Demonstrations	6
8.2	GEPA 学習設定	7

9

評価結果と参照値との比較

7

9.1

到達点

7

9.2

学習された知識（進化後指示文）

8

10

再現手順

8

11

既知の限界と改良余地

8

1 目的とスコープ

課題：各最適化問題は自然言語の要件と JSON のインスタンスデータを持つ。目標は、LLM に `solve(instance)` という Python 関数を生成させ、その関数が出力する解の目的値が**参照値（既知の良解）**以上となる問題数を最大化することである。

手法：DSPy の `ChainOfThought` でコードを生成し、安全なサンドボックスで実行して制約充足・目的値を採点する。GEPA (Genetic-Pareto Prompt Optimization) が採点フィードバックを用いて生成用の指示文 (Instructions) を進化させる。

2 つの動作モード：

- **参照ありモード（既定）：**報酬に参照値を用い、「参照一致 / 参照超え」を高く評価する。
- **参照フリーモード (`--no-reference`)：**参照値を報酬・フィードバックから完全に排除し、強化学習的に「自ら得た解の改善」のみを報酬とする。参照値の無い新規問題への適用を想定した動作。参照値は**最終結果の解析にのみ使用する**。

2 実行環境と依存関係

項目	値
Python	3.11 (3.9 は multiprocessing/GEPA で不具合)
DSPy	3.3.0b1
LLM	Qwen3.6-27B (OpenAI 互換エンドポイント)
エンドポイント	<code>http://10.81.105.102:11434/v1</code> (<code>src/lm_config.py</code>)
temperature	0.2
max_tokens	8192
timeout	1800 秒
環境変数	<code>PYTHONIOENCODING=utf-8</code>

許可インポート（生成コード内）：`math, random, heapq, itertools, collections, functools, typing, bisect, operator, json, copy, re, ortools, scipy, pulp, networkx, numpy`。**禁止：**`os, sys, subprocess, socket, pickle, requests, urllib, importlib, ctypes`。LLM エンドポイントは `src/lm_config.py` の `configure_qwen_default()` で切替可能。別環境で再現する場合は `api_base/model` を差し替える。

3 データ仕様

3.1 入力ファイル形式

各問題は 1 つの JSON ファイル (prob_001.json ~ prob_100.json, 計 100 問)。トップレベルのキーは以下。

キー	型	内容
id	int	問題 ID
name	str	問題名
domain	str	分野 (例: スケジューリング)
math_type	str	数理タイプ (例: 動的計画法)
difficulty	str	難易度
split	str	参考用の train/test ラベル
description	str	自然言語の問題文
requirements	dict	objective (目的文) と constraints
instance	dict	実データ (solve に渡る dict そのもの)
reference_solution	dict	参照解 (objective_value と解構造)

参照値は `reference_solution.objective_value`。**目的文**は `requirements.objective` (例: 「総遅延時間の最小化」) で、最適化方向 (最小化/最大化) と目的フィールド特定に用いる。`core_type` はデータには無く, `domain + math_type` から `data_loader.core_type_from_v3()` が生成する (例: スケジューリング_動的計画法)。

3.2 訓練/テスト分割

`data_loader.load_and_split_stratified(data_dir, n_train=40, n_test=20, seed=42)` により, 100 問から 40 訓練 / 20 テストを層化抽出する。

1. 全レコードを `core_type` でグループ化。
2. 各グループ内を `seed=42` で決定的にシャッフル。
3. ラウンドロビンでテスト → 訓練の順に 1 問ずつ割当て, 両分割に `core_type` 多様性を確保。

`seed` 固定により分割は完全再現可能であり, 参照あり/参照フリーの比較は同一のテスト集合上で行われる。

3.3 要件テキストの構築

`data_loader` は各問題を `dspy.Example` 化する際, `requirement_builder` で LLM 入力用の要件テキストを組み立てる。これには (a) カテゴリ (`core_type`), (b) 問題文, (c) 入出力フォーマット, (d) インスタンスの生 JSON, (e) 参照解の構造 (トップレベルのフィールド名・形状) が含まれる。**参照解の「数値」は参照フリーモードでは隠蔽されるが, 「構造 (キー名)」は問題仕様として常に提示する (§7)**。推論時と学習時で同一 builder を使い, 整合性を保つ。

4 パイプライン構成

ファイル	役割
src/data_loader.py	データ読込・層化分割・Example 化・要件構築
src/requirement_builder.py	LLM 入力要件テキスト生成
src/signatures.py	3 つの Signature (指示文) 定義
src/modules.py	AlgorithmGenerator パイプライン
src/metrics_v3.py	評価メトリック本体
src/gepa_feedback_v3.py	GEPA 用フィードバック生成
src/best_known.py	既知最良/セルフベースライン管理
src/utils/safe_exec.py	サンドボックス実行
src/utils/feasibility.py	制約充足検査
src/utils/scorer.py	目的値計算補助
src/lm_config.py	LLM エンドポイント設定
train_gepa_v3.py	学習・評価エントリポイント

4.1 アルゴリズム生成 (AlgorithmGenerator)

src/modules.py の AlgorithmGenerator は dspy.Module で、3 つの ChainOfThought を持つ。

- parse_instance (ParseInstance) : instance dict を型安全にパースする補助コードを生成。
- generate (GenerateOptimizationAlgorithm) : solve(instance) 本体を生成。GEPA が進化させる中心。
- improve (ImproveAlgorithm) : フィードバックを受けてコードを改良 (反復救済用)。

forward() では、安定性のため汎用の安全アクセサ (get_list/get_dict/get_scalar) を固定 parse_code として注入し、実際のキーは要件テキスト中の生 JSON から LLM が読み取る設計とした (instance を with_inputs に含めないことで GEPA コンパイル時の入力を要件テキストに一本化)。LLM 出力からは strip_code_fence() で `'''python ... '''` コードを抽出する。

4.2 安全実行 (safe_exec)

生成コードは multiprocessing (spawn) で親から隔離実行する。実行時間上限、禁止インポートの遮断、例外捕捉を行い、戻り値 (解 dict) または例外情報を返す。spawn を用いるため、全実行ハーネスは if __name__ == '__main__': 下に置く必要がある。

5 Signature 指示文 (初期)

生成の中心 GenerateOptimizationAlgorithm (src/signatures.py) の初期指示文は、最小限の **HARD RULES** に絞る。これは GEPA が進化させる「土台」であり、薄いほど進化が起きやすい。

1. トップレベル関数はちょうど 1 つ : def solve(instance):。
2. 解を return する (print 不可)。戻り値は要件の「参照解構造」節と同じトップレベルのフィールド名で、非空でなければならない。
3. 空スケジュール・空ルート・cost=0 を返さない (score=0.1 で棄却)。

4. 実行時間上限 30 秒 (OR-Tools はソルバ時間制限を設定)。
5. アルゴリズム本体を try/except で包み、失敗時は非空解を返す貪欲ヒューリスティックにフォールバック。
6. **SYNTAX SAFETY**：括弧の対応を徹底し、`model.Add(expr) for x in items` のような末尾ジェネレータを禁止 (実ループを使用)。OR-Tools 構文起因の `exec.error` を抑制する。

入力フィールドは `requirement` (生 JSON と参照解構造を含む), `core_type`, `parse_code`。出力フィールドは `algorithm_code`。

進化後の指示文は `src/signatures.py` には無く、学習成果物 `compiled_program_v3_gepa*.json` の `generate.predict.signature.instructions` に格納される (§8, §9)。再現時はこの JSON を `--program-path` で読み込めば学習済み指示文で評価できる。

6 評価メトリック

評価は `evaluate.algorithm_v3()` (`src/metrics_v3.py`) が担う。処理順は**実行** → **構造検証** → **制約検証** (`feasibility`) → **目的値算出** → **採点**。

6.1 目的フィールドの自動検出

参照解が問題ごとに全く異なる構造を持つため、解 `dict` の中の「どの数値フィールドが目的値か」を `_select_objective_field()` が汎用スコアで推定する。

条件	スコア
<code>min_ / max_</code> 接頭辞	+5
目的文トークンとの一致 (日英対応)	+4
集約接頭辞 <code>total_ / peak_ / expected_ / best_ / optimal_</code>	+2
解にキーが存在	+1
値がコンテナ要素数と一致 (構造的カウント)	-4
カウント的な名前	-1
出現順タイブレーク	$-\text{idx} \times 0.01$

最大スコアのフィールドを目的値とみなす。最適化方向は `min_ / max_` 接頭辞 → 目的文の「最大/最小」 → キーワードの優先順で決定する。複数目的候補を持つ問題群で 10/10 の正解を確認済み。参照解が全く無い問題向けに、解 `dict` から自己参照で目的フィールドを推定する `generic_ref_guided_cost` のフォールバックも実装している。

6.2 スコア計算 (`compute_v3_score`)

コストは「小さいほど良い」に正規化される (最大化問題は符号反転)。`best_known` を過去最良コスト, `anchor` を参照アンカー (参照ありモードでは `reference_cost`, 参照フリーでは初回実行可能コスト=セルフベースライン) とする。

■(1) 退化解の検出 `anchor > 0` かつ `cost < anchor \times 0.05` の場合、空/無効解の疑いとして `suspicious_zero`, `score = 0.3`。

■(2) ベーススコア (`best_known` との比)

- `best_known` が未設定: `base = 1.0 (first_valid)`。
- 通常: `ratio = best_known/cost`, `base = clip(2.0(ratio - 0.5), 0, 2)`。`ratio > 1.01` で `improved`, `< 0.99` で `worse`, その間は `similar`。

■(3) 参照ボーナス (参照ありモードのみ) `reference_cost > 0` のとき,

- $|\text{cost} - \text{reference_cost}| < 10^{-6}$: `exact_match`, `base = max(base + 0.2, 1.5)`。
- $0 < \text{cost} < \text{reference_cost}$: 改善率 $r = (\text{ref} - \text{cost})/|\text{ref}|$ に対しボーナス $\min(0.5, 0.2 + 0.3r)$, `base = max(base + bonus, 1.5)`, `status beat_reference`。

参照フリーモードではこのブロックを完全にスキップする。

■(4) `best_known` 改善ボーナス $0 < \text{cost} < \text{best_known}$ で $+0.15$, 1-10% の微小改善に比例ボーナス。最終 `score = min(base, 2.5)`。

6.3 status 一覧

status	意味	概略スコア
<code>exact_match</code>	参照値と一致 (参照ありのみ)	≥ 1.5
<code>beat_reference</code>	参照値を厳密改善 (参照ありのみ)	≥ 1.5
<code>improved/new_best</code>	既知最良を更新	> 1.0
<code>first_valid</code>	初の実行可能解	1.0
<code>similar</code>	最良とほぼ同等	~ 1.0
<code>worse</code>	最良より悪い有効解	$0 < s < 1$
<code>suspicious_zero</code>	退化解の疑い	0.3
<code>infeasible</code>	制約違反	0.0
<code>exec_error</code>	実行時例外	0.0
<code>parse_error</code>	出力構造不正	0.0

6.4 フィードバック生成

`gepa_feedback_v3` (`src/gepa_feedback_v3.py`) は評価結果を自然言語フィードバックに変換して GEPA に渡す。`classify_exec_error()` が例外を種別 (`import/syntax/timeout/type` 等) に分類し, `summarize_ref_resolution()` が参照解のトップレベルフィールド構造を要約する。これによりキー名不一致や構文エラーが指示文へ学習可能となる。`gap_to_reference` (参照とのギャップ) は常に計算されるが, 参照フリーモードでは解析専用であり報酬・フィードバックには使われない。

7 参照フリーモードの厳密化

参照値を学習から完全排除するには, 値が混入し得る 3 経路すべてを断つ必要がある (`train_gepa_v3.py --no-reference`)。

1. 報酬: `compute_v3_score(use_reference=False)`。`exact_match/beat_reference` ボーナスと参照ベースの退化検出を無効化し, セルフベースライン (初回実行可能コスト, `best_known.set_baseline_if_absent()`) を正規化アンカーに用いる。
2. `best_known` 初期化: 参照値からの seeding を停止し, 参照シード済み `best_known.jsonl` を読み込

まず、空の `best_known_noref.jsonl` から開始。

3. フィードバック文: `set_use_reference(False)` と `summarize_ref_solution(hide_values=True)` により、「この値以上を狙え」という数値目標行を削除。出力スキーマ（フィールド名・形状）のみ残す。

コスト測定は両モードで同一に保つため、目的フィールドの特定には参照解の**構造**を引き続き用いる（値は使わない）。これにより比較は「参照値を報酬に使うか否か」だけを操作した対照実験となる。出力スキーマの提示は「解答」ではなく「問題仕様」であり、参照解答のリークには当たらない。

8 Demonstrations と学習設定

8.1 Demonstrations

`dspy.GEPA` は `Instructions` のみを進化させ `few-shot` 例は保持するため、既知の良コードを `program.generate.predict.demos` に手動付与する (`build_demonstrations()`, `train_gepa_v3.py`)。 `exact_match` 型 (`prob.001`, `DP`, 約 1.6KB) と `beat_reference` 型 (`prob.021`, `VRP`, 約 9KB) の 2 実例を常在させ、「完全一致を狙う」ケースと「参照を上回る」ケースの双方を例示する。 `--no-demos` で無効化可能。

8.2 GEPA 学習設定

項目	値
訓練/テスト	40 / 20 (<code>--n-train/--n-test</code>)
breadth	6 (<code>--breadth</code>)
depth	8 (<code>--depth</code>)
valset	訓練の 1/3 (40 なら 13, 残り 27 が <code>train_only</code>)
demos	2

GEPA が指示文を進化させるには、経験上「(a) ベース Signature を薄く」「(b) 予算を数十評価以上」「(c) `valset` を訓練の 1/3 以上」の 3 条件が必要である。

9 評価結果と参照値との比較

100 問・40 訓練 / 20 テスト (`seed=42`), `breadth 6`・`depth 8`, `demos 2` の結果を示す。比較にはスコア式に依存しない**事後指標**を用いる：実行可能かつ実コストを持つ結果について `cost≤ref` を「参照以上」、`cost<ref` を「厳密超え」と数える (`exec_error/infeasible` 等は除外)。この指標はモード非依存で `per-problem` に一致する。

表 1 参照あり vs 参照フリー（同一テスト分割・事後指標）

指標	参照あり		参照フリー	
	訓練	テスト	訓練	テスト
参照以上 (<code>cost≤ref</code>)	22/40	7/20	20/40	7/20
厳密超え (<code>cost<ref</code>)	9/40	1/20	8/40	1/20
有効解	29/40	12/20	37/40	19/20
<code>exec_error</code> (テスト)		1		1

9.1 到達点

- テスト参照超えは両モードで完全一致（7/20，厳密 1/20）。参照値を報酬から取り除いても参照超え件数は 100% 維持された。
- 参照フリーの方が有効解率が高い（テスト 12/20 → 19/20）。制約充足＋自己改善の報酬が「まず正しく動く解」を強く促した結果である。
- `exec_error` はテスト 1 件に抑制（HARD RULE #6 の効果）。
- 平均スコアはモード間で報酬スケールが異なるため直接比較しない。公平な軸は上記の事後指標である。

9.2 学習された知識（進化後指示文）

参照ありモードの進化後 `generate` 指示文は約 17KB に達し，末尾に「Critical Learning from Previous Failures」節を形成する。実失敗から学んだ参照解キー名の具体例（下表）を多数含み，V3 の「参照解構造が問題ごとに全く異なる」困難に直接効いた。

問題タイプ	正しい目的/出力キー	誤りやすいキー
Bottleneck Assignment	<code>min.bottleneck</code>	<code>bottleneck_time</code>
Parallel Machine Sched.	<code>machine_assignment</code> (Job ID)	Machine ID キー
Staff Rostering	<code>roster</code> (Staff ID)	Time slot キー
Shortest Path (P2P)	<code>shortest_distance</code>	<code>distance</code>

10 再現手順

1. Python 3.11 環境に DSPy 3.3.0b1 と許可ライブラリ (`ortools`, `scipy`, `pulp`, `networkx`, `numpy`) を導入。
2. `src/lm.config.py` の `api_base/model` を自環境の LLM エンドポイントに合わせる。
3. データ 100 問 (`prob.001.json`~`prob.100.json`) を配置し，`train_gepa_v3.py` の `--data-dir` で指す。
4. `PYTHONIOENCODING=utf-8` を設定。

■参照ありモードの学習＋評価

```
python311 train_gepa_v3.py --n-train 40 --n-test 20 \  
  --breadth 6 --depth 8 --data-dir <DATA_DIR>  
# 出力: compiled_program_v3_gepa_phaseE.json,  
# evaluation_results_v3_gepa_phaseE.json
```

■参照フリーモードの学習＋評価

```
python311 train_gepa_v3.py --no-reference --n-train 40 --n-test 20 \  
  --breadth 6 --depth 8 --data-dir <DATA_DIR>  
# 出力: compiled_program_v3_gepa_phaseF_noref.json,  
# evaluation_results_v3_gepa_phaseF_noref.json,  
# data/best_known_noref.jsonl
```

■学習済みプログラムの再評価のみ

```
python311 train_gepa_v3.py --eval-only \  
  --program-path compiled_program_v3_gepa_phaseE.json --data-dir <DATA_DIR>
```


注意：長時間・detach プロセスの標準出力を `Select-Object -First N` 等の早期終了パイプに通さないこと（プロセスが停止する）。ログはファイルヘリダイレクトして別途参照する。学習は本環境で約 3 時間強を要する。

11 既知の限界と改良余地

- テスト 20 問には運要素が残る。複数 seed 分割での平均評価が望ましい。
- 時系列・複合構造（`trains` 等）で目的フィールド検出が誤ることがある。目的文の構文解析による補強が有効。
- 参照フリーの `new_best` は空レジストリ由来で膨らむため、`status` 単独では意味を持たない。事後指標が真の信号である。
- 進化後指示文が 17KB と肥大化。タイプ別の階層化・要約が課題。
- 完全スキーマレス化（目的文のみから目的フィールドを自己識別）はフォールバック実装済みで、次段階の検証対象である。